

Platform Architecture for CIS Level 1 Audit Readiness

Container Supply Chain, Infrastructure-as-Code Governance, and Configuration Drift Control

Prepared by: Lead Platform Architect | Status: For Review

This is the reference architecture for closing the three gaps raised ahead of our CIS Level 1 assessment: unverified images reaching production, changes made outside infrastructure code, and no defined remediation window for drift.

Container Supply Chain Design

This section defines how a container image travels from developer commit to running production workload; it matters to the audit because CIS Level 1 requires that only images of verified provenance are admitted to the cluster.

The chain begins in CI/CD. On merge to main, the pipeline builds the image and pushes it to Azure Container Registry (ACR) at <REGISTRY_URL> under an immutable tag <IMAGE_TAG>, so the artifact an auditor inspects is the artifact that ran.

Immediately after a successful push, the pipeline runs a signing stage using Notation, the Notary v2 client. The signing certificate is held in Azure Key Vault at <KEY_VAULT_NAME>, and the pipeline authenticates using workload identity federation, so no key material sits in the pipeline definition or on the build agent. Notation writes the signature to the registry as an OCI artifact linked to the image manifest, so it travels with the image rather than living in a store that can fall out of sync.

Verification happens at admission. Ratify runs in-cluster as an external data provider for Gatekeeper, the OPA-based admission controller. The Gatekeeper validating webhook resolves each image reference to its digest and asks Ratify to verify it; Ratify retrieves the linked signature from ACR and checks it against the trusted certificate. If verification fails, or no signature exists, admission is denied and the pod is never scheduled. An unsigned image, or one from a registry outside the allowlist, cannot reach production — which closes the first gap.

The constraint below is illustrative of that enforcement:

```
apiVersion: constraints.gatekeeper.sh/v1beta1
kind: K8sRequiredSignatures
metadata:
  name: enforce-acr-signatures
spec:
  parameters:
    trustedRegistries:
      - "<REGISTRY_URL>"
    requiredSigners:
      - "<SIGNING_CERT_NAME>"
```

Signature verification proves where an image came from, not what it may do once running. We therefore apply the PSS Baseline profile through Pod Security Admission (PSA) in enforce mode on every production namespace, blocking privilege escalation, host namespace sharing, and hostPath mounts.

PSS Baseline enforced via PSA is the minimum workload hardening CIS Level 1 expects; Restricted is a roadmap item.

IaC Module Registry Structure

This section defines the versioned module library all environments are provisioned from; it matters to the audit because ClickOps is only preventable once there is one approved, reviewable path for creating infrastructure.

Every module lives in one governed repository with a fixed, labelled layout:

```
modules/           # approved infrastructure modules
  network/         # VNets, subnets, NSGs
    main.bicep     # module definition
    version.json   # semantic version
  storage/         # storage accounts, endpoints
    main.bicep
    version.json
  policies/        # Azure Policy definitions
    tagging/       # ownership and cost tags
    policy.json    # deny on missing tags
```

Each module folder carries its own version.json, and modules are versioned semantically as MAJOR.MINOR.PATCH. A MAJOR bump is any breaking change to the module contract: removing or renaming a required parameter, changing a naming convention, or altering a property that forces resource replacement. A MINOR bump is additive and backward compatible — a new optional parameter with a safe default, or a new output. A PATCH bump changes no interface: defect fixes and corrected defaults. Environments pin to an exact version, so every upgrade is a deliberate pull request with a reviewable trail.

Governance is embedded in the module rather than left to the caller, as in this parameterised stub:

```
param location string = '<LOCATION>'
param ownerTag string = '<TAG_OWNER>'

var governedTags = {
  owner: ownerTag
  costCentre: '<COST_CENTRE_ID>'
  managedBy: 'platform-iac'
}
```

The concrete control is a mandatory Azure Policy assignment at subscription scope, defined in policies/tagging/policy.json, denying creation of any resource missing the owner and costCentre tags. Because the deny evaluates at the resource provider, an engineer creating a resource by hand in the portal is rejected exactly as a malformed pipeline run is. It makes provisioning outside the approved pattern fail rather than relying on teams to choose the modules.

Drift Detection and Remediation Workflow

This section defines how we discover that a deployed environment no longer matches declared state and the process commitment for closing the gap; it matters to the audit because CIS Level 1 expects a bounded, owned remediation window rather than best-effort cleanup.

Detection runs nightly, as a scheduled comparison per subscription and environment during the low-change window. It compares declared state in the module repository against the configuration currently deployed, runs under a reader-scoped identity, and is strictly read-only. Automatic correction is excluded deliberately: silently reverting a change made during an incident would be worse than the drift itself.

A drift finding records the resource identifier, the subscription and environment, the property that differs, the declared and observed values, the timestamp first seen, and the owning team — derived from the mandatory owner tag above, which makes assignment immediate rather than an investigation.

The 48-hour service level is a commitment about how the team behaves, not a metric a tool calculates:

1. At hour zero the finding is opened as a ticket and assigned automatically to the team named in the resource owner tag; findings never sit in an unassigned queue.
2. Within 48 hours the owner closes it one of two ways. Revert: if unintended, redeploy the pinned module version and the environment returns to declared state. Adopt: if legitimate, raise a pull request amending the module or environment parameter file so declared state includes the change; the finding closes when that merge is applied.
3. At 24 hours without acknowledgement the ticket escalates to the owning team lead. At 48 hours without closure it escalates to the platform engineering lead and goes on the exception register with a named accepting owner and a review date, so nothing ages out silently.
4. Repeat drift on the same resource within a rolling 30-day window escalates as a pattern rather than an incident, since it usually means the module lacks a parameter teams need.

The architecture makes this timeline achievable: ownership tags remove the search for an accountable engineer, version pinning reduces a revert to one pipeline run, and read-only detection means no step waits on approval.

Policy Enforcement Summary

This section maps each in-scope CIS Level 1 requirement to the control that satisfies it, so a reviewer can trace requirement to implementation in one pass.

CIS Level 1 Requirement	Architectural Control
Image provenance	ACR + Notation (Notary v2) signing, verified by Ratify via Gatekeeper
Workload hardening	Pod Security Admission enforcing the PSS Baseline profile
Approved provisioning	Versioned IaC module registry with embedded policy hooks

CIS Level 1 Requirement	Architectural Control
Resource accountability	Mandatory owner and cost-centre tags enforced by Azure Policy (deny)
Drift detection	Nightly read-only comparison of declared state against deployed state
Drift remediation	48-hour SLA with named owner, revert-or-adopt closure, and escalation

Every control above is enforced at admission, at the resource provider, or through a named-owner process, so none depends on individual discipline. Evidence is retrievable without privileged access: signature artifacts in the registry, policy definitions in source control, and drift ticket history.